



Guía de aprendizaje

Integración y aplicación de pruebas unitarias en microservicios con JUnit

Exp 2 – Semana 6

Desarrollo Backend II (PBY2202)

Facilitador disciplinar: Ányelo Castellón Ríos

Asesor par: Ignacio Villarroel Sánchez

Índice

Introducción a la semana.....	4
Resultado de aprendizaje.....	5
Conceptos relevantes.....	5
Preguntas activadoras.....	6
Actividad.....	6
Reforcemos conceptos previos antes de JUnit.....	7
JUnit: definición de Pruebas unitarias y su rol en microservicios.....	7
Configuración básica de JUnit.....	9
Generación y lectura de resultados con JUnit.....	13
Análisis de resultados de pruebas unitarias.....	14
Interpretación de reportes de pruebas.....	15
Métricas y calidad de las pruebas.....	17
Identificación de Patrones de fallos.....	18
Estrategias para mejorar la calidad a partir del análisis.....	21
Refactorización orientada por pruebas.....	21
Ajuste y mejora de casos de prueba.....	24
Integración Continua (CI) y Monitoreo Permanente.....	25
Ejemplo práctico.....	26
Paso a paso del ejemplo.....	27
Creación de Pruebas unitarias.....	30
Ejecución de Pruebas Unitarias.....	32
Análisis de resultados.....	32
Cierre de la semana.....	33

Referencias.....	34
Glosario.....	35
Apuntes.....	38

Introducción a la semana



En esta semana, nos enfocaremos en el análisis profundo de los resultados de las pruebas unitarias. A partir de la información generada por herramientas estándar en la industria, como JUnit, aprenderás a interpretar reportes, métricas y patrones de fallos, de modo que puedas evaluar si el código cumple con los requisitos y se mantiene en línea con las expectativas de

calidad.

Además de aprender cómo leer e interpretar los resultados de las pruebas, aprenderás a identificar áreas de mejora y aplicar los hallazgos encontrados para optimizar tanto el código como las propias pruebas, lo que evitará que errores no detectados se propaguen a otras etapas del desarrollo y/o producción.

Los contenidos clave que encontrarás en esta guía son:

- Interpretación de reportes de pruebas unitarias generados por JUnit
- Identificación de fallos, métricas y tendencias en los resultados
- Acciones de mejora a partir del análisis, como refactorización del código o ajuste de casos de prueba

Resultado de aprendizaje

El estudiante será capaz de:

RA2. Crea pruebas unitarias en los microservicios, usando tecnologías como JUnit, para asegurar calidad del desarrollo elaborado, de acuerdo a requerimientos y estándares de la industria.

Indicador de logro:

IL5. Configura un entorno de pruebas unitarias en microservicios, utilizando diversas herramientas de testing y frameworks, garantizando la cobertura y automatización de las pruebas.

IL6. Diseña pruebas unitarias para microservicios utilizando JUnit, cumpliendo con los requerimientos especificados y estándares de la industria.

IL7. Ejecuta pruebas unitarias en microservicios desarrollados utilizando JUnit y herramientas actuales, garantizando la calidad y funcionalidad del código.

IL8. Realiza análisis de los resultados de las pruebas unitarias en microservicios para verificar el cumplimiento de los requerimientos y mejorar la calidad del desarrollo.

Conceptos relevantes

JUnit	Análisis de resultados	Métricas
Reportes	Fallos	Requerimientos
Calidad	Refactorización	Mejora continua

Preguntas activadoras

- ¿De qué manera el análisis de los reportes generados por JUnit te permite identificar problemas de calidad en el código y priorizar su resolución?
- ¿Cómo influye la interpretación de las métricas de prueba (tiempo de ejecución, porcentaje de éxito, cobertura) en la toma de decisiones para mejorar la calidad del servicio?
- ¿De qué forma reconocer patrones de fallos en las pruebas unitarias puede orientarte hacia una refactorización más efectiva de módulos específicos en el entorno de microservicios?
- ¿Cómo integrarías el análisis continuo de los resultados de las pruebas unitarias en tu flujo de desarrollo para asegurar una mejora constante en la calidad del producto?

Actividad

En esta sexta semana, realizarás una actividad sumativa grupal (2 a 3 integrantes), llamada "Integrando, aplicando y analizando resultados de pruebas unitarias en microservicios con JUnit", donde deberás integrar un entorno de pruebas unitarias en microservicios utilizando JUnit, garantizando la cobertura y automatización.

Reforcemos conceptos previos antes de JUnit

Antes de profundizar en el uso de JUnit y su aplicación en microservicios, es importante repasar algunos conceptos clave que servirán como base para entender y aprovechar mejor las herramientas y estrategias que se abordarán en esta guía. Estos conceptos conectan directamente con los pasos que realizaremos más adelante y ayudan a garantizar una transición fluida hacia el uso práctico de JUnit.



JUnit: definición de Pruebas unitarias y su rol en microservicios

¿Qué son las Pruebas unitarias?

Las pruebas unitarias son un tipo de verificación enfocada en evaluar la funcionalidad de las partes más pequeñas y aisladas del código (por lo general, una función o método específico). Su principal propósito es asegurar que cada unidad de software cumpla con los requerimientos esperados de manera independiente. De esta forma, se detectan errores desde etapas tempranas del desarrollo, lo que reduce costos y esfuerzo de corrección en fases posteriores.

Beneficios de las Pruebas unitarias en el Ciclo de desarrollo

- **Detección temprana de errores:** localizar problemas en fases tempranas facilita su corrección con menor impacto en tiempo y recursos.
- **Fomento de la confianza en el código:** contar con una batería sólida de pruebas unitarias permite realizar cambios y refactorizaciones con la certeza de no introducir errores de forma inadvertida.
- **Documentación viva:** las pruebas unitarias sirven como una especificación técnica, ya que describen la funcionalidad esperada del código y pueden utilizarse como referencia para nuevos desarrolladores.

El rol de las pruebas unitarias en la arquitectura de microservicios

Los microservicios dividen la aplicación en componentes independientes, cada uno con su propia lógica de negocio y recursos. Este paradigma favorece la escalabilidad, el despliegue continuo y la resiliencia. Sin embargo, también requiere un mayor esfuerzo de pruebas. En este contexto, las pruebas unitarias resultan cruciales para:

- **Asegurar integridad interna:** antes de integrar un microservicio con otros componentes, es fundamental verificar que cada pieza cumpla su cometido de manera aislada.
- **Promover la independencia:** al garantizar el correcto funcionamiento de cada microservicio individualmente, se facilita la identificación de errores y su corrección sin afectar el sistema en su totalidad.
- **Facilitar la escalabilidad:** una arquitectura de microservicios acompañada de un conjunto sólido de pruebas unitarias permite agregar o modificar servicios con mayor agilidad, reduciendo el riesgo de generar regresiones.

Configuración básica de JUnit



Como ya sabemos, JUnit es un framework de pruebas unitarias para Java que entrega una forma estructurada de crear, ejecutar y reportar pruebas, permitiendo a los desarrolladores verificar el comportamiento del código de forma automatizada. Las principales ventajas de JUnit incluyen:

- **Simplicidad de uso:** cuenta con anotaciones e infraestructura sencillas, lo que facilita su implementación en proyectos existentes.
- **Integración con herramientas de desarrollo:** JUnit se integra fácilmente con IDEs (como IntelliJ, Eclipse) y servidores de integración continua (como Jenkins o GitHub Actions), favoreciendo un flujo de desarrollo continuo.
- **Reportes claros y métricas de cobertura:** su compatibilidad con herramientas de reporte y análisis permite visualizar resultados y métricas, indispensables para el posterior análisis de la calidad del software.

Ahora que hemos revisado rápidamente sus ventajas, podemos comenzar a utilizar JUnit en un proyecto Java. Para hacerlo, sigue los siguientes pasos:

Paso 1 - Dependencia: incluir la dependencia de JUnit (por ejemplo, JUnit 5) en el archivo de configuración del proyecto (pom.xml en Maven, build.gradle en Gradle).

Figura 1

Dependencia JUnit 4

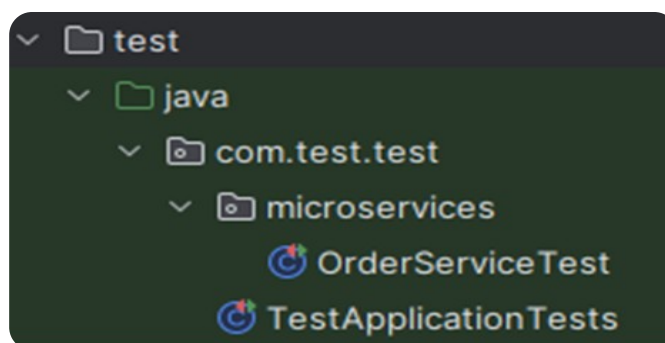
```
<dependency>
  <groupId>junit</groupId>
  <artifactId>junit</artifactId>
  <version>4.13.2</version>
  <scope>test</scope>
</dependency>
```

Nota. Configuración de Maven para usar JUnit. JetBrains. (2024). *IntelliJ IDEA*. (2024.2.4). [Software]. <https://www.jetbrains.com/idea/download/?section=windows>

Paso 2 - Estructura de pruebas: ubicar las clases de prueba en un directorio dedicado (generalmente src/test/java).

Figura 2

Estructura de pruebas en el proyecto



Nota. Organización de pruebas unitarias en el paquete com.test.test.microservices. JetBrains. (2024). *IntelliJ IDEA*. (2024.2.4). [Software]. <https://www.jetbrains.com/idea/download/?section=windows>

Paso 3 - Creación de casos de prueba: cada caso de prueba se define como un método anotado con `@Test`. Dentro de ese método se implementan aserciones para validar el comportamiento esperado.

Figura 3

Ejemplo de Test para un cálculo del total en un pedido

```
package com.test.test.microservices;
import static org.junit.jupiter.api.Assertions.assertEquals;
import org.junit.jupiter.api.Test;

public class OrderServiceTest {

    @Test
    public void testCalculateTotal() {
        // Arrange: Creamos un pedido con items conocidos
        Order order = new Order();
        order.addItem(new Item( nombre: "Producto A", precio: 1000.0));
        order.addItem(new Item( nombre: "Producto B", precio: 2000.0));
        order.addItem(new Item( nombre: "Producto C", precio: 500.0));

        OrderService orderService = new OrderService();

        // Act: Calculamos el total
        double resultado = orderService.calculateTotal(order);

        // Assert: Verificamos el cumplimiento del requerimiento
        // El total esperado es 3500.0
        assertEquals( expected: 3500.0, resultado,
            message: "El total del pedido no coincide con la suma esperada de los items.");
    }
}
```

Nota. Verifica que el total del pedido coincida con la suma de los precios de los ítems añadidos. JetBrains. (2024). *IntelliJ IDEA*. (2024.2.4). [Software].

<https://www.jetbrains.com/idea/download/?section=windows>

Paso 4 - Ejecución: las pruebas se ejecutan directamente desde el IDE o mediante comandos de construcción (ej: mvn test o gradle test). Al final, JUnit presenta un reporte indicando qué pruebas pasaron, cuáles fallaron y por qué.

Figura 4

Ejecución de pruebas y compilación

```
[INFO] Results:
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 13.387 s
[INFO] Finished at: 2024-06-01T12:27:38+05:30
[INFO] -----
```

Nota. Todas las pruebas se ejecutaron con éxito, sin fallos ni errores, y la compilación fue exitosa. JetBrains. (2024). *IntelliJ IDEA*. (2024.2.4). [Software].

<https://www.jetbrains.com/idea/download/?section=windows>

Una vez que hayas ejecutado tu prueba, es momento de analizar los resultados entregados por JUnit, lo que veremos a continuación.

Generación y lectura de resultados con JUnit



Una vez que las pruebas han sido ejecutadas, JUnit puede entregar los resultados de las pruebas en distintos formatos, tales como texto plano, HTML o XML, dependiendo de la configuración empleada. Sin embargo, más allá de las diferencias en los formatos, los resultados incluidos en los reportes consideran:

- **Listado de pruebas exitosas y fallidas:** permite identificar rápidamente qué partes del código presentan problemas.
- **Mensajes de error y stack traces:** ofrecen pistas sobre la causa raíz del fallo, acelerando la localización del error.
- **Métricas adicionales (con herramientas externas):** integrando JUnit con herramientas de cobertura como JaCoCo, por ejemplo, es posible visualizar el porcentaje de líneas cubiertas, ayudando a evaluar la eficacia de las pruebas y la necesidad de agregar más casos.

Los conceptos que hemos revisado en la sección sientan las bases para comprender y analizar los resultados de las pruebas unitarias en microservicios. Dominar estos fundamentos, desde la definición de pruebas unitarias hasta la configuración básica de JUnit y la interpretación de reportes iniciales, es un paso esencial para avanzar hacia el análisis profundo de resultados y la mejora continua de la calidad del software.

Análisis de resultados de pruebas unitarias

Ahora que hemos repasado los conceptos fundamentales relacionados con las pruebas unitarias —su importancia, uso de herramientas como JUnit y la relevancia de cubrir los principales escenarios— daremos el siguiente paso para centrarnos específicamente en la necesidad de realizar análisis de los resultados de las pruebas unitarias en microservicios para verificar el cumplimiento de los requerimientos y mejorar la calidad del desarrollo.

En esta nueva sección, profundizaremos en:

- **Cómo interpretar reportes y métricas** que arrojan las pruebas unitarias,
- **Identificación de patrones de fallos** para distinguir entre defectos reales y problemas en las propias pruebas, y
- **Estrategias de mejora continua**, como refactorización y ajustes en el set de pruebas, que se derivan de dicho análisis.

De esta manera, pasamos de una visión teórica de las pruebas unitarias a un enfoque práctico de **evaluación y refinamiento** del código, y sentamos las bases para implementar un ciclo de mejora continua que asegure la calidad y confiabilidad de cada microservicio.

Interpretación de reportes de pruebas

Los reportes generados tras la ejecución de pruebas unitarias muestran el estado general del software, presentando información sobre el total de pruebas realizadas, resultado (aprobadas o fallidas), errores detectados y tiempo de ejecución. Estos reportes se generan de forma automática y suelen estar disponibles en formatos legibles y para herramientas de integración continua.

Figura 5

Test en consola

```
mvn test

-----
T E S T S
-----
Running HelloWorldTest
Hello World
Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.079 sec

Results :

Tests run: 1, Failures: 0, Errors: 0, Skipped: 0

[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 02:14 min
[INFO] Finished at: 2020-04-21T14:06:46Z
[INFO] -----
```

Nota. Todas las pruebas se ejecutaron con éxito, sin fallos ni errores, y la compilación fue exitosa. JetBrains. (2024). *IntelliJ IDEA*. (2024.2.4). [Software].

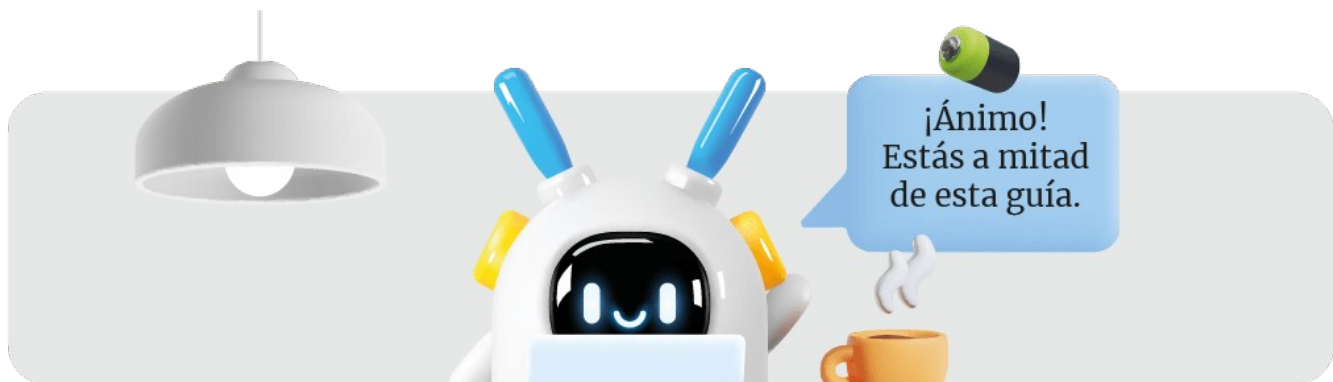
<https://www.jetbrains.com/idea/download/?section=windows>

Elementos clave de los reportes

- Cantidad total de pruebas ejecutadas.
- Número de pruebas exitosas y fallidas.
- Mensajes de error y *stack traces* asociados a cada fallo.
- Tiempo total y por prueba.
- Enlaces o secciones detalladas para profundizar en el análisis de cada caso.

Tabla 1*Ejemplo de información en un reporte*

Elemento	Descripción	Ejemplo
Pruebas totales	Cantidad de casos ejecutados	30 pruebas en total
Exitosas	Casos sin fallas	28 aprobadas
Fallidas	Casos con error	2 con AssertionError
Mensajes de error	Detalles del fallo	expected <100> but was <90>
Tiempo de ejecución	Duración de cada prueba	testCalcularTotal (5ms)



Métricas y calidad de las pruebas



Cobertura de código

La cobertura indica qué porcentaje del código es ejercitado por las pruebas. Una cobertura alta sugiere mayor probabilidad de detectar defectos, aunque no garantiza calidad absoluta. De igual forma, una cobertura baja señala áreas del código sin validar, potencialmente riesgosas.

Número de pruebas superadas y fallidas

La relación entre pruebas aprobadas y fallidas aporta una visión general de la estabilidad del código. Un alto número de fallos puede reflejar problemas lógicos, mala configuración de pruebas o casos no contemplados en el diseño del software.

Tiempo de ejecución de las pruebas

Pruebas excesivamente lentas pueden indicar ineficiencias en el código, dependencias externas no simuladas o diseños que no favorecen el aislamiento de componentes. Reducir el tiempo de ejecución optimiza el flujo de desarrollo e integración continua.

Inestabilidad de las pruebas (test flakiness)

Las pruebas inestables, que en ocasiones son exitosas y en otras fallan sin cambios en el código, generan desconfianza en el suite de pruebas. Esta situación suele deberse a factores externos, entornos no deterministas, datos variables y/o dependencias mal simuladas.

Tabla 2*Tabla de métricas y acciones sugeridas*

Métrica	Descripción	Acción Potencial
Cobertura 80%	80% del código se ejercita en pruebas	Añadir pruebas para casos no cubiertos
2 fallos en 30 Pruebas	Falta de robustez en ciertas funciones	Revisión del módulo fallido o la prueba
Tiempo total 300ms	Pruebas lentas señalan ineficiencias	Refactorizar para mejorar rendimiento
Prueba inestable	Resultado impredecible	Asegurar datos deterministas y entornos aislados

Identificación de Patrones de fallos

Patrones recurrentes

Si una misma prueba o un conjunto de pruebas relacionadas fallan sistemáticamente, existe un patrón de fallo. Este patrón puede apuntar a defectos crónicos en el código o a un mal diseño de las pruebas.

Falsos positivos y falsos negativos

- **Falso positivo:** la prueba indica un error, pero el código está correcto. Esto suele ocurrir cuando las condiciones de prueba están mal definidas o las aserciones no reflejan el requerimiento real.
- **Falso negativo:** la prueba no detecta un error que sí existe en el código. Esto ocurre cuando las pruebas no cubren adecuadamente el caso fallido o las aserciones son insuficientes.
-

Tabla 3

Falso positivo y falso negativo

Tipo de error	Escenario de ejemplo	Causa probable	Cómo solucionarlo
Falso Positivo	La prueba indica que esMayorDeEdad(17) debe retornar false, pero curiosamente la prueba marca “error” al afirmar que debería ser true.	Aserción mal planteada: el test espera true cuando el requerimiento real es false (la persona tiene 17 años).	Corregir la aserción para que coincida con la lógica de negocio real (17 años -> menor de edad).
		Datos de prueba incorrectos o mal interpretados.	Ajustar la documentación o los datos usados en la prueba.
Falso Negativo	La prueba pasa exitosamente cuando esMayorDeEdad(20) retorna false, sin detectar que la lógica está mal implementada y en realidad debería retornar true.	Condición insuficiente: la prueba no contempla adecuadamente que 20 años es, por definición, mayor de edad.	Revisar la lógica del método y las condiciones de la prueba.
		Aserciones incompletas o lógica demasiado laxa.	Asegurar que el test verifique correctamente los límites y valores esperados (≥ 18 años).

Distinción entre defecto real y problema en la prueba

Para diferenciar si el origen del fallo está en el código o en la propia prueba, se debe analizar el mensaje de error, el *stack trace* y las condiciones del test. Ajustar datos de entrada, revisar aserciones o aislar dependencias puede ayudar a aclarar si se trata de un defecto funcional o de una falla en el escenario de prueba.

Tabla 4

Patrones de fallo

Tipo de fallo	Descripción	Ejemplo	Acción
Defecto real del código	El método no cumple lo esperado	calcTotal() retorna 90 en vez de 100	Refactorizar la lógica
Problema en la prueba	Las aserciones no reflejan el requerimiento	Aserción espera 100, requerimiento es 90	Ajustar la prueba acorde al requerimiento
Falso Positivo	Error reportado sin razón	Aserción incorrecta provoca fallo	Corregir las aserciones
Falso Negativo	No se detecta un error existente	Prueba no contempla caso límite	Agregar casos faltantes

La interpretación de reportes, el análisis de métricas y la identificación de patrones de fallo permiten comprender a fondo el desempeño del software que estamos probando. Esto facilita la toma de decisiones informadas para mejorar la calidad del código, asegurar el cumplimiento de los requerimientos y avanzar hacia la creación de pruebas unitarias que permitan asegurar la calidad del desarrollo acorde a los requerimientos y estándares de la industria, así como también realizar un análisis que permita verificar el cumplimiento de los requerimientos y mejorar la calidad del desarrollo.

Estrategias para mejorar la calidad a partir del análisis



Una vez que hayas realizado el análisis de los resultados de las pruebas unitarias, el siguiente paso es aplicar acciones concretas para elevar la calidad del desarrollo. Las estrategias que presentamos a continuación son tres:

- Refactorización orientada a pruebas.
- Ajuste y mejora de casos de prueba.
- Integración Continua (CI) y Monitoreo Permanente.

Cada una de estas estrategias parten de la información obtenida del análisis previo, transformándola en ajustes al código, a las pruebas y a la infraestructura de integración continua.

Refactorización orientada por pruebas

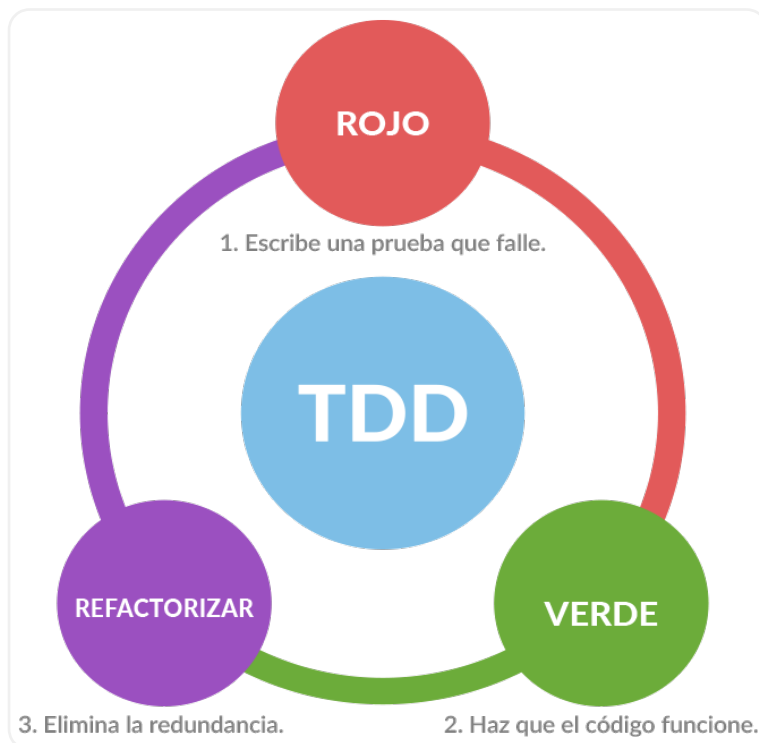
Consiste en realizar mejoras al código basadas en la información obtenida del análisis de resultados de las pruebas. Estas mejoras no alteran la funcionalidad final, sino que buscan optimizar la estructura interna, la claridad y la mantenibilidad del software. Dado que las pruebas unitarias ofrecen retroalimentación constante, la refactorización se guía por datos objetivos sobre calidad y rendimiento.

En este punto, puedes utilizar la metodología Test Driven Development (TDD), que se caracteriza porque en lugar de escribir códigos y probar su funcionamiento, se escriben primero las pruebas y luego se desarrolla el código.

Para ello, se sigue un ciclo iterativo al que se conoce como **rojo-verde-refactorizar** o **red green refactor**. El primer paso es escribir la prueba que define una funcionalidad o un comportamiento deseado. Después, se implementa el código que se necesita para pasar la prueba. En último lugar, se refactoriza dicho código para mejorar su calidad o estructura sin cambiar su comportamiento.

Figura 6

Ciclo TDD: Rojo - Verde - Refactorizar



Nota. La imagen representa el ciclo del Desarrollo Dirigido por Pruebas (TDD), que consta de tres fases: Rojo, Verde y Refactorizar. Rocabado, M. (2018). *Introducción al Desarrollo Guiado por Pruebas (TDD)*. <https://mindwaresrl.com/2015/10/18/introduccion-al-desarrollo-guiado-por-pruebas-tdd/>

La ventaja que ofrece el TDD frente a otros enfoques, es que el código se prueba desde el comienzo. Por tanto, si hay errores, se pueden identificar con rapidez para corregirlos.

Acciones clave en la refactorización

- **Eliminar código redundante:** remover funciones o variables duplicadas para reducir complejidad.
- **Simplificar lógica compleja:** dividir métodos muy extensos en otros más pequeños y legibles.
- **Mejorar nomenclatura:** usar nombres descriptivos para métodos, clases y variables.
- **Adoptar buenas prácticas de diseño (SOLID):** asegurar que la arquitectura resulte flexible y escalable.

Tabla 5

Posibles problemas identificados

Problema detectado	Acción de refactorización	Resultado esperado
Lógica repetida en 3 clases	Extraer la lógica a un método común	Menor duplicación y mantenimiento
Métodos con más de 100 líneas	Dividir en sub-métodos lógicos	Mayor legibilidad y simplicidad
Variables con nombres genéricos	Renombrar variables según su función	Código más claro e intuitivo

Ajuste y mejora de casos de prueba

No solo el código productivo se beneficia del análisis, también las propias pruebas. Ajustar y mejorar las pruebas garantiza que reflejen adecuadamente los requerimientos, cubran escenarios omitidos, eliminen redundancias y mantengan un equilibrio entre cantidad y calidad.

Acciones clave para mejorar las pruebas

- **Agregar escenarios omitidos:** incluir casos límite, valores nulos, datos extremos.
- **Eliminar redundancias:** quitar pruebas duplicadas o que no aporten información nueva.
- **Clarificar aserciones:** utilizar aserciones descriptivas para facilitar la interpretación de fallos.
- **Revisar Mocks/Stubs:** asegurar que las dependencias simuladas se ajusten al comportamiento real esperado.

Tabla 6

Posibles mejoras identificadas

Problema en las pruebas	Acción de mejora	Resultado esperado
Falta de pruebas con datos nulos	Agregar casos de entrada nula	Detección temprana de fallos
2 pruebas idénticas	Eliminar una prueba redundante	Menor tiempo de ejecución
Aserciones poco específicas	Usar aserciones claras (ej. <code>assertEquals</code>) con mensajes descriptivos	Mayor facilidad de diagnóstico

Integración Continua (CI) y Monitoreo Permanente

La integración continua permite ejecutar automáticamente las pruebas con cada cambio en el código. Esto habilita un monitoreo constante, detección temprana de regresiones y una respuesta rápida a los problemas. De esta manera, el análisis de resultados no se limita a un evento aislado, sino que se transforma en un proceso continuo.

Buenas prácticas en CI

- **Ejecución automática de pruebas:** configurar pipelines que corran las pruebas en cada commit.
- **Análisis de cobertura y código estático:** integrar herramientas para medir cobertura (JaCoCo) y calidad (SonarQube).
- **Notificaciones tempranas:** activar alertas en canales de mensajería cuando las pruebas fallan.
- **Paneles de control (Dashboards):** visualizar métricas de calidad en tiempo real.

Tabla 7

Herramientas y sus funciones

Herramienta	Función	Beneficio
Jenkins/GitHub Actions	Pipeline de CI para ejecutar pruebas	Detección inmediata de fallos
JaCoCo	Reporte de cobertura de código	Identifica áreas no testeadas
SonarQube	Análisis estático y calidad continua	Mejora integral del código

Ejemplo práctico

Siguiendo la dinámica de las semanas anteriores, crearemos un microservicio de ejemplo y le agregaremos las pruebas unitarias con JUnit. Para ello, en el siguiente enlace encontrarás lo necesario para llevar a cabo este ejercicio: https://ava.duoc.cl/bbcswebdav/xid-5381910_1

En particular, pondremos el foco en un microservicio sencillo (por ejemplo, un microservicio de “Productos”), que expone un servicio REST para administrar el listado de productos en una base de datos simulada.

- Estructura de directorios (ejemplo con Gradle o Maven).
- Dependencias de JUnit.
- Asegúrate de tener la dependencia de JUnit (versión 5 recomendada) en tu pom.xml (Maven) o build.gradle (Gradle).
- Si usas Spring Boot, también tendrás las dependencias de spring-boot-starter-test que incluyen JUnit.

Paso a paso del ejemplo

Paso 1: Crea la entidad Product

En la carpeta “entity” definimos una clase Product, que representará al producto dentro de nuestro microservicio:

Figura 7

Definición de clase Product

```
public class Product { no usages
    private Long id; 3 usages
    private String name; 3 usages
    private Double price; 3 usages

    // Constructor
    public Product(Long id, String name, Double price) { no usages
        this.id = id;
        this.name = name;
        this.price = price;
    }
}
```

Nota. Creación del entity Product con su constructor JetBrains. (2024). *IntelliJ IDEA*. (2024.2.4). [Software]. <https://www.jetbrains.com/idea/download/?section=windows>

Paso 2: Crea la clase de servicio ProductService

Aquí implementaremos la lógica principal (muy básica en este ejemplo) para administrar nuestra lista de productos:

Figura 8

Creación de Service ProductService

```
@Service no usages
public class ProductService {
    private List<Product> productList = new ArrayList<>(); 4 usages

    public Product addProduct(Product product) { no usages
        productList.add(product);
        return product;
    }

    public List<Product> getAllProducts() { no usages
        return productList;
    }

    public Product getProductById(Long id) { no usages
        return productList.stream() Stream<Product>
            .filter( Product p -> p.getId().equals(id))
            .findFirst() Optional<Product>
            .orElse( other: null);
    }

    public void clearProducts() { no usages
        productList.clear();
    }
}
```

Nota. Service de product y sus métodos. JetBrains. (2024). *IntelliJ IDEA*. (2024.2.4). [Software]. <https://www.jetbrains.com/idea/download/?section=windows>

Paso 3: Crea el controlador ProductController

En un proyecto con Spring Boot, el controlador podría lucir así:

Figura 9

Creación del Controller ProductController

```
@RestController
@RequestMapping("/products")
public class ProductController {

    @Autowired
    private ProductService productService;

    @PostMapping
    public Product createProduct(@RequestBody Product product) {
        return productService.addProduct(product);
    }

    @GetMapping
    public List<Product> getAllProducts() {
        return productService.getAllProducts();
    }

    @GetMapping("/{id}")
    public Product getProductById(@PathVariable Long id) {
        return productService.getProductById(id);
    }
}
```

Nota. RestController y sus métodos Get y Post. JetBrains. (2024). *IntelliJ IDEA*. (2024.2.4). [Software]. <https://www.jetbrains.com/idea/download/?section=windows>

**Importante**

Para propósitos de la demostración, estamos omitiendo aspectos como persistencia real en base de datos y configuración de Spring. El objetivo es centrarnos en la lógica de negocio y su prueba unitaria.

Ahora, revisemos a partir de lo aprendido con el ejemplo, como se realizaron:

- La creación de pruebas unitarias,
- La ejecución de pruebas unitarias, y
- El análisis de los resultados.

Creación de Pruebas unitarias

Para diseñar nuestras pruebas unitarias, partimos analizando los métodos del servicio y definiendo qué se espera como resultado correcto. De este modo, cubrimos los diferentes escenarios y validamos si el servicio funciona correctamente bajo distintos supuestos.

- **Clase de Test: ProductServiceTest**

En la carpeta test, creamos la clase de prueba correspondiente:

Figura 10

Creación del Test de entity Product

```
package cl.backend2.pruebasunitarias;

import cl.backend2.pruebasunitarias.entity.Product;
import cl.backend2.pruebasunitarias.service.ProductService;
import org.junit.jupiter.api.*;

import java.util.List;

import static org.junit.jupiter.api.Assertions.*;

class ProductServiceTest {

    private ProductService productService; 11 usages

    @BeforeEach
    void setUp() {
        productService = new ProductService();
    }

    @AfterEach
    void tearDown() {
        productService.clearProducts();
    }

    @Test
    @DisplayName("Agregar producto y verificar que se almacene correctamente")
    void testAddProduct() {
        Product product = new Product( id: 1L, name: "Laptop", price: 999.99);
        Product result = productService.addProduct(product);

        assertNotNull(result);
        assertEquals( expected: 1L, result.getId());
        assertEquals( expected: "Laptop", result.getName());
        assertEquals( expected: 999.99, result.getPrice());
    }
}
```

Nota. Test de product y sus distintas formas de métodos de prueba. JetBrains. (2024). *IntelliJ IDEA*. (2024.2.4). [Software]. <https://www.jetbrains.com/idea/download/?section=windows>

En cada método de prueba se diseña un escenario para verificar el correcto funcionamiento de la capa de servicio. Así diseñamos pruebas unitarias contemplando diferentes condiciones (producto existente, producto inexistente, lista vacía, etc.).

Ejecución de Pruebas unitarias

Para ejecutar las pruebas unitarias, podemos utilizar distintas herramientas:

- IDE: IntelliJ IDEA, Eclipse o VSCode, simplemente dando clic derecho en la clase de prueba y seleccionando “Run Test”.
- Terminal (Maven): `mvn test`
- Terminal (Gradle): `gradle test`

Esto generará un reporte del resultado de cada test, indicando cuántas pruebas se ejecutaron, cuántas pasaron y cuántas fallaron (si corresponde).



Importante

Asegúrate de tener configurado tu proyecto con las dependencias de prueba adecuadas (JUnit 5, por ejemplo) para que la ejecución sea exitosa.

Análisis de resultados

Una vez ejecutadas las pruebas, revisamos:

- **Cantidades de pruebas exitosas y fallidas:** si alguna prueba falla, revisamos la causa en el log o reporte generado (puede estar relacionado con la lógica de negocio o con supuestos incorrectos en la prueba).
- **Cobertura de pruebas (opcionalmente):** herramientas de cobertura como JaCoCo pueden ayudarnos a revisar qué partes del código se están probando.
- **Mejoras al servicio:** si detectamos fallas lógicas en la capa de negocio, corregimos la lógica y volvemos a ejecutar los tests hasta que pasen satisfactoriamente.

Cierre de la semana

A lo largo de esta semana, hemos explorado en profundidad el análisis de los resultados de las pruebas unitarias en entornos de microservicios, comprendiendo cómo esta etapa es clave para mejorar de forma continua la calidad del desarrollo.

Iniciamos con la interpretación de reportes y métricas, lo que nos permitió identificar fallos, evaluar el grado de cobertura y detectar patrones que señalaban áreas de mejora tanto en el código como en las propias pruebas. Posteriormente, abordamos estrategias concretas: refactorización guiada por los hallazgos, ajuste y optimización de los casos de prueba y la incorporación de prácticas de integración continua para monitorear permanentemente la calidad.

Al extraer información valiosa de cada ejecución, se adquiere la capacidad de reaccionar con agilidad ante problemas, proponer acciones correctivas informadas y mantener el producto alineado con los estándares de la industria. De este modo, las pruebas unitarias, combinadas con su análisis y mejora continua, se consolidan como un pilar esencial para garantizar la confiabilidad, mantenibilidad y evolución constante del software.



Referencias

- Bechtold, S., Brannen, S., Link, J., Merdes, M., Philipp, M., de Rancourt, J., & Stein, C. (s.f.). *jUnit 5 User Guide*. <https://junit.org/junit5/docs/current/user-guide/>
- Mockito. (s.f.). *Mockito*. <https://site.mockito.org/>

Glosario

En el siguiente glosario encontrarás términos esenciales en la seguridad de aplicaciones backend y el uso de Spring Security. Te invitamos a leerlos y familiarizarte con todos ellos.

Término	Definición
Prueba Unitaria	Verificación de la funcionalidad de la unidad más pequeña del código (generalmente una función o método) para asegurar que opere según lo esperado.
JUnit	Framework estándar en Java para la creación, ejecución y reporte de pruebas unitarias, facilitando la verificación automática de la lógica del código.
Microservicio	Componente independiente de una arquitectura distribuida, responsable de una función específica, que puede ser desarrollado, probado y desplegado por separado.
Resultado de Aprendizaje (RA)	Declaración de lo que el estudiante debe ser capaz de hacer al finalizar una experiencia de aprendizaje, orientando el diseño de contenidos y evaluaciones.
Indicador de Logro (IL)	Evidencia concreta que permite medir el grado de logro de un resultado de aprendizaje, mostrando el avance o cumplimiento de un objetivo formativo.
Cobertura de Código	Métrica que indica el porcentaje del código ejercitado por las pruebas, evidenciando partes no testeadas que podrían ocultar defectos.
Asersión (Assertion)	Condición utilizada dentro de una prueba unitaria para verificar que un resultado obtenido coincide con el resultado esperado.

Test Flakiness (Inestabilidad) Situación donde una prueba a veces pasa y a veces falla sin cambios en el código, dificultando la confianza en el conjunto de pruebas.

Refactorización Proceso de mejorar la estructura interna del código sin modificar su comportamiento externo, incrementando su mantenibilidad y legibilidad.

Integración Continua (CI) Práctica de ejecutar automáticamente las pruebas y análisis de calidad del código con cada modificación, detectando problemas de forma temprana.

Pipeline de CI Secuencia automatizada de pasos (construcción, pruebas, análisis, despliegue) que permite un monitoreo permanente de la calidad y funcionalidad del software.

Mock Objeto simulado que imita el comportamiento de una dependencia externa, facilitando la ejecución de pruebas unitarias aisladas.

Stub Tipo de objeto simulado más simple que un mock, proporcionando datos predefinidos para las pruebas, sin lógica compleja.

Informe de Pruebas (Reporte) Documento generado automáticamente tras la ejecución de las pruebas, detallando casos exitosos, fallidos, tiempo y mensajes de error.

Métrica Valor cuantitativo que permite medir aspectos específicos de la calidad y rendimiento del código y las pruebas (ej. cobertura, tiempo de ejecución).

Patrones de Fallos Tendencias o repeticiones en los errores detectados por las pruebas, que orientan a la identificación de problemas crónicos en código o pruebas.

Falso Positivo Situación en que una prueba señala un error inexistente en el código,

comúnmente debido a condiciones de prueba incorrectas.

Falso Negativo Caso en el que una prueba no detecta un defecto real del código, dejando pasar un error sin reportarlo.

Mejora Continua Enfoque sistemático para refinar y optimizar el código, las pruebas y los procesos de desarrollo, asegurando una calidad creciente a lo largo del tiempo.

Requerimiento Necesidad o condición que el software debe cumplir, definida antes del desarrollo, y validada a través de pruebas unitarias y análisis de resultados.

Apuntes

This image shows a single sheet of white paper with horizontal ruling lines. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.



Reservados todos los derechos Fundación Instituto Profesional Duoc UC. No se permite copiar, reproducir, reeditar, descargar, publicar, emitir, difundir, de forma total o parcial la presente obra, ni su incorporación a un sistema informático, ni su transmisión en cualquier forma o por cualquier medio (electrónico, mecánico, fotocopia, grabación u otros) sin autorización previa y por escrito de Fundación Instituto Profesional Duoc UC. La infracción de dichos derechos puede constituir un delito contra la propiedad intelectual.